Advanced Programming

Java Collections Framework

Collections

- A *collection* is an object that groups multiple elements into a single unit.
- *Vectors, Lists, Stacks, Sets, Dictionaries, Trees, Tables, etc.*
- Promotes software reuse
- Reduces programming effort
- Increases program speed and quality
- Benefits from *polymorphic algorithms*
- Uses *generics*

Collections Framework

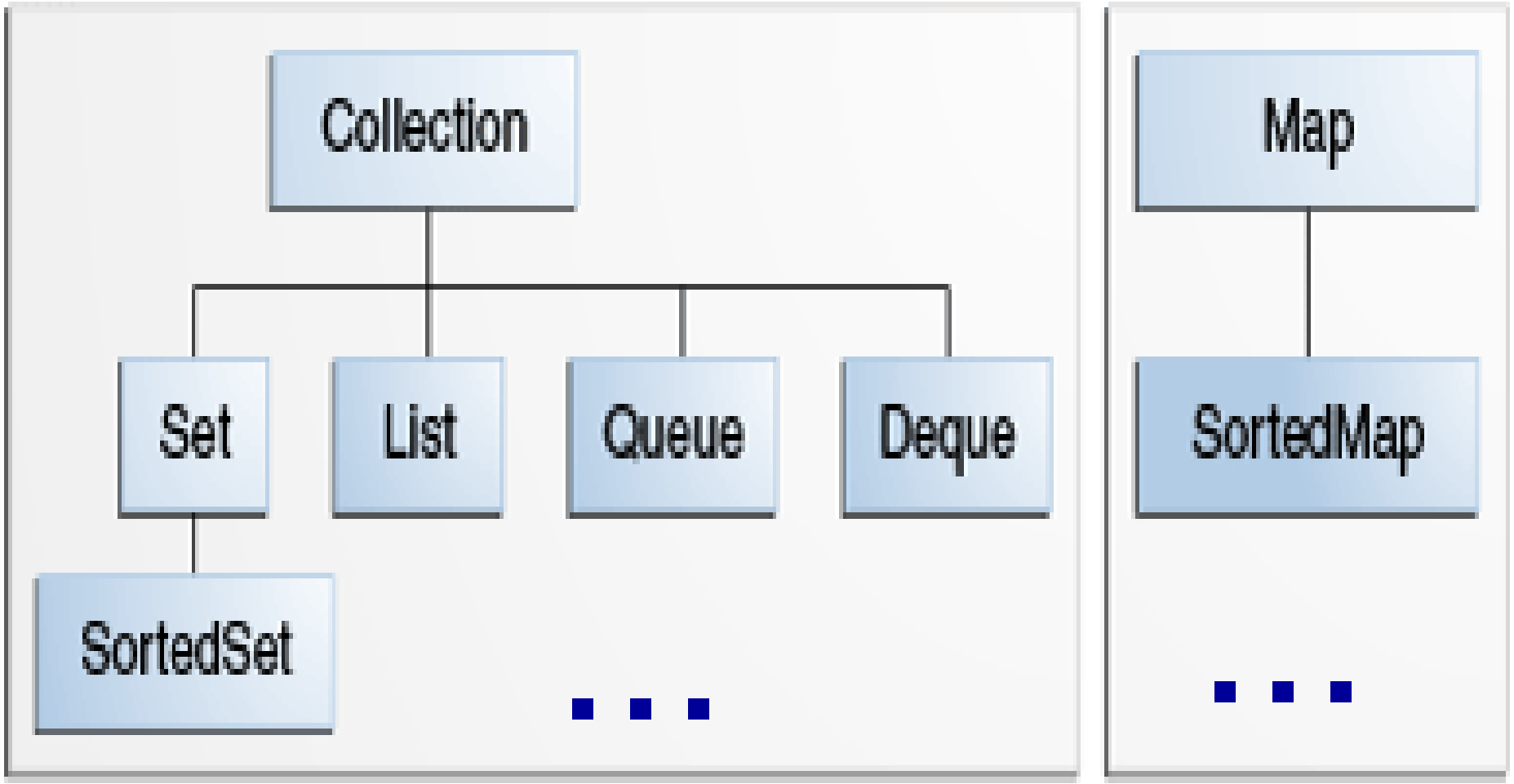
- Interface
- Abstract class
- Concrete implementation

List

AbstractList

ArrayList
LinkedList
Vector

The Core Collection Interfaces



Implementations

Interfața	Hash	Array	Tree	Linked	Hash+Linked
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList Vector		LinkedList	
Queue		ArrayBlocking Queue		LinkedList, ConcurrentLin kedQueue	
Deque		ArrayDeque		LinkedList, LinkedBlockin gDeque	
Map	HashMap Hashtable		TreeMap		LinkedHashMap

The *Collection* interface

- A collection represents a group of objects (elements)
- Some collections are *indexed*, others are not.
- Some collections allow *duplicate* elements, others do not.
- Some are *ordered* and others unordered.
- Some are *thread-safe*, others are not.
- All collections are *iterable*:

  Iterator pattern

  - `public interface Collection<E> extends Iterable<E>`
- Dedicated subinterfaces: `List, Set, Queue, etc.`
- Many (indirect) implementing classes.

Common operations

```
public interface Collection<E> extends Iterable<E> {

    int size();
    boolean isEmpty();
    boolean contains(Object o);
    Iterator<E> iterator();

    Object[] toArray();
    <T> T[] toArray(T[] a);
    default <T> T[] toArray(IntFunction<T[]> generator) {
        return toArray(generator.apply(0));
    }

    boolean add(E e);
    boolean remove(Object o);
    boolean containsAll(Collection<?> c);          ← Inclusion
    boolean addAll(Collection<? extends E> c); ← Union
    boolean removeAll(Collection<?> c);            ← Difference
    boolean retainAll(Collection<?> c);            ← Intersection
    void clear();
    ...
}
```

Lists: ArrayList, LinkedList

- A *list* is an indexed collection (sequence).
- Each element has a position (integer), starting from 0. Just like an array.
- Duplicates **are** allowed.
- A list can be implemented in various ways

```
List<String> list = new ArrayList<>();
```

```
List<Number> list = new LinkedList<>();
```

Bad practice:
Using implementations as reference types:

```
ArrayList<String> list = new ArrayList<>();
```

Using raw generics:

```
List list = new ArrayList();
```

Using a *List*

```
List<String> list = new ArrayList<>();

list.add("a");

list.add("b");

System.out.println(list); // [a, b];


assert list.size() == 2;

assert list.get(0).equals("a");

assert list.get(1).equals("b");

assert list.indexOf("a") == 0;


list.remove("b");

assert !list.contains("b");

assert list.contains("a");
```

To enable assertions use the JVM option **-ea**

java -ea MyApp

indexOf, contains use the *equals* method of the elements!

Under the Hood

- **ArrayList**<E> implements **List<E>**

```
private static final int DEFAULT_CAPACITY = 10;
Object[] elementData;
private int size;
```

- **LinkedList**<E> implements **List<E>**

```
int size = 0;
Node<E> first;
Node<E> last;
```

```
private static class Node<E> {
    E item;
    Node<E> next;
    Node<E> prev;
    …
}
```

The importance of *equals*

- Methods like: *contains, indexOf, remove, etc.* rely on the *equals* method:

  public boolean contains(Object o)

  returns true if this list contains the specified element. More formally, returns true if and only if this list contains at least one element e such that (o==null ? e==null : o.equals(e)).

- Source code (in ArrayList, approximative):

```
for (int i = 0, n = size(); i < n; i++) {
  if (o.equals(elementData[i])) {
    return true;
  }
}
return false;
```

Iterating Over a List

- As an indexed collections

```
for (int i=0, n=list.size(); i < n; i++ ) {
    System.out.println( list.get(i) );
}
```

- *Iterator* and *Enumeration*

```
for (Iterator it = list.listIterator(); it.hasNext(); ) {
  System.out.println(it.next());
  it.remove();
}
```

- *for-each*

```
List<Student> students = new ArrayList<>();
...
for (Student student : students) {
    student.setGrade(10);
}
```

ArrayList or *LinkedList*?

```
public class TestList {
    private final static int N = 100_000;
    public void testAdd(List<Integer> list) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            list.add(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Add: " + (t2 - t1));
    }
    public void testGet(List<Integer> list) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            list.get(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Get: " + (t2 - t1));
    }
    public void testRemove(List<Integer> list) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            list.remove(0);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Remove : " + (t2 - t1));
    }
    public void runTests(List<Integer> list) {
        testAdd(list); testGet(list); testRemove(list);
    }
    public static void main(String args[]) {
        TestList app = new TestList();
        app.runTests(new ArrayList<>());
        app.runTests(new LinkedList<>());
    }
```

	ArayList	LinkedList
add	6 ms	8 ms
get	3 ms	4320 ms
remove	868 ms	6 ms

Conclusion: Choosing a certain implementation depends on the nature of the problem being solved.

The *Set* interface

- A collection that contains no duplicate elements (models the mathematical set abstraction).

  More formally, sets contain no pair of elements e1 and e2 such that *e1.equals(e2)*, and at most one null element.
- Sets are not indexed! We use *iterators* or *for-each* in order to traverse the set.
- A set can be implemented in various ways

```
Set<String> list = new HashSet<>();
```

```
Set<Number> list = new TreeSet<>();
```

Under the Hood

- **HashSet**<E> implements Set<E>

  ```
  HashMap<E,Object> map;
  ```

  Uses the *hashCode* of an Object.

  This class offers **constant time** performance for the basic operations (add, remove, contains and size), assuming the hash function disperses the elements properly among the buckets.

- **TreeSet**<E> implements Set<E>

  ```
  private NavigableMap<E,Object> m; //a specialized SortedMap
  ```

  The elements are sorted using their natural ordering, or by a *Comparator* provided at set creation time

  This implementation provides guaranteed **log(n) time** cost for the basic operations (add, remove and contains).

Using a *Set*

```
Set<String> set = new HashSet<>();
set.add("b"); set.add("a"); set.add("a");
System.out.println(set); // [a, b]
```

No guarantees about order.

```
assert set.size() == 2;
assert set.get(0).equals("a");
```

```
set.remove("b");
assert set.contains("a");
```

indexOf, contains use the *equals* method of the elements.

```
for(String s : set) {
    System.out.println(s);
}
```

ArrayList or *HashSet*?

```
public class TestSet {
    final static int N = 100_000;
    public void testAdd(Collection<Integer> collection) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            collection.add(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Add: " + (t2 - t1));
    }
    public void testIterate(Collection<?> collection) {
        long t1 = System.currentTimeMillis();
        for (Object obj : collection) {
            obj.toString(); //do something
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Iterate: " + (t2 - t1));
    }
    public void testContains(Collection<?> collection) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            collection.contains(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Contains: " + (t2 - t1));
    }
    public void runTests(Collection<Integer> collection) {
        testAdd(collection); testIterate(collection); testContains(collection);

    }
    public static void main(String args[]) {
        TestSet app = new TestSet();
        app.runTests(new ArrayList<>());
        app.runTests(new HashSet<>());
    }
}
```

	ArayList	HashSet
add	6 ms	26 ms
iterate	51 ms	12 ms
contains	3997 ms	7 ms
Memory	low	high

Conclusion: Choosing a certain implementation depends on the nature of the problem being solved.

The *Queue* interface

- A queue is a first-in-first-out **(FIFO)** linear collection;

  It has a *head* and a *tail*.

- Provides additional *insertion*, *extraction*, and *inspection* operations.

  `add, remove, element` (throw exceptions)

  `offer, poll, peek` (return special values)

- Example

```
Queue<String> q = new LinkedList<>();
q.offer("a"); // same as q.add
q.offer("b"); // adds elements at the tail
assert q.peek().equals("a"); //returns the head
assert q.poll().equals("a"); //returns and removes
assert q.peek().equals("b");
```

The *PriorityQueue* class

- A queue based on a priority **heap**. The elements of the priority queue are ordered according to their natural ordering, or by a *Comparator.* The *head* of the queue is *the least element* with respect to the ordering.
- The implementation provides **O(log(n))** time for the enqueing and dequeing methods, **linear time** for the *remove* and *contains* methods; and **constant time** for the retrieval methods (*peek, element, size*).
- Example:

```
PriorityQueue<Integer> pQueue = new PriorityQueue<>();

pQueue.add(3); pQueue.add(1); pQueue.add(2);

assert pQueue.peek().equals(1);
```

*Stack*s and *Deque*s

- The *Stack* class represents a last-in-first-out (**LIFO**) stack of objects. Obsolete, just like *Vector*.
- ***Deque*** is a linear collection that supports element insertion and removal at both ends ("double ended queue").
- Deques can be used to implement LIFO collections:

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); //same as addFirst
stack.push(2);
assert stack.peekFirst().equals(2); //same as peek
assert stack.peekLast().equals(1);
```

The *Map* interface

- An object that maps keys to values.
- It is similar to describing a *dictionary.*
- `public interface Map<K, V> { … }`
- A map cannot contain duplicate keys; each key can map to at most one value.
- Example: `Map<Country, City>`

Key		Value
Romania	→	Bucharest
France	→	Paris

- Implementations:

```
Map<Country, City> capitals = new HashMap<>();
Map<Word, List<Definition>> dict = new TreeMap<>();
```

Using a *Map*

```
Map<String, String> map = new HashMap<>();
map.put("Romania", "Bucharest");
map.put("France", "Paris");
System.out.println(map);
//Romania=Bucharest, France=Paris}

assert map.get("France").equals("Paris");
assert map.containsKey("Romania");

for(String country : map.keySet()) {
    System.out.println(map.get(country));
}
```

Under the Hood

- *HashMap* can achieve an average time complexity of O(1) for the *put* and *get* operations and space complexity of O(n).
- Instead of iterating over all its elements, *HashMap* attempts to calculate the position of a value based on its key.
- HashMap stores elements in so-called buckets and the number of buckets is called *capacity*.

```
Node<K,V>[] table;

Set<Map.Entry<K,V>> entrySet;
```

- The *index* in the *table* array is computed using the *hashCode* method of the key object.
- Inside a bucket, values are stored either in a *list* (for less than 8 elements) or in *a balanced tree.* The data structure changes dynamically. The performance of iterating: O(log |bucket|).

HasMap.get

```
final Node<K,V> getNode(int hash, Object key) {

  Node<K,V>[] tab; Node<K,V> first, e; int n; K k;

  if ((tab = table) != null && (n = tab.length) > 0 &&
      (first = tab[(n - 1) & hash]) != null) {
      if (first.hash == hash && // always check first node
          ((k = first.key) == key
            || (key != null && key.equals(k))))
          return first;
      if ((e = first.next) != null) {
          if (first instanceof TreeNode)
            return ((TreeNode<K,V>)first).getTreeNode(hash, key);
          do {
            if (e.hash == hash &&
              ((k = e.key) == key
                || (key != null && key.equals(k))))
               return e;
          } while ((e = e.next) != null);
      }
  }
  return null;
}
```

hashCode and *equals*

```
public class Player {
    private String name;
    // ...

    @Override
    public int hashCode() {
        int hash = 7;
        hash = 67 * hash + Objects.hashCode(this.name); //example
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null) return false;
        if (!(obj instanceof Player)) return false;
        Player other = (Player) obj;
        return Objects.equals(this.name, other.name);
    }

    // ...
}
```

When a *hashCode*() comparison returns false, the *equals*() method must also return false.

The importance of *equals*

Assume there is another property *Integer id*;

```
public boolean equals(Object obj) {

    if (this == obj) return true;
    if (obj == null) return false;
    if (!(obj instanceof Player)) return false;
    Player pers = (Player) other;
    return Objects.equals(this.id, other.id);
 }
```

```
Map<Player, String> map = new HashMap<>();
map.put(new Player("Messi"), "PSG");
map.put(new Player("Ronaldo"), "Manchester United");

System.out.println(map);
```

{Messi=Manchester United}

Polymorphic Algorithms

`java.util.Collections`

- `sort`
- `shuffle`
- `binarySearch`
- `reverse`
- `fill`
- `copy`
- `min`
- `max`
- `swap`
- `enumeration`
- `unmodifiable`*`CollectionType`*

```
List<String> immutablelist = Collections.unmodifiableList(list);
immutablelist.add("Oops...?!");
```

What DesignPattern?

- `synchronized`*`CollectionType`*

var

Type inference for local variables

- The explicit type can be replaced by the reserved type name **var** for local variable declarations that have initializers.

```
String str = "Hello";
```

```
var str = "Hello World";
```

- Explicit vs. Implicit, Verbosity vs. Readability
- Examples

```
var stringList = List.of("a", "b", "c");

var personList = new ArrayList<Person>();
for(var person : personList) { }

var companyToEmployees = new HashMap<String, List<String>>();
for (var entry: companyToEmployees. entrySet()) {
     var employees = entry.getValue();
}

var itemQueue = new PriorityQueue<>(); // PriorityQueue<Object>
var x = null;
```